

第二章 语法分析 (自底向上的分析技术)

冯洋

南京大学计算机学院
fengyang@nju.edu.cn

提纲

- 上下文无关文法
- 语法分析技术
 - 自顶向下
 - 自底向上
 - 概述
 - 移进-规约技术
 - LR语法分析技术
 - 简单LR技术
 - LR技术: SLR, LALR,.....
- 语法分析器生成工具

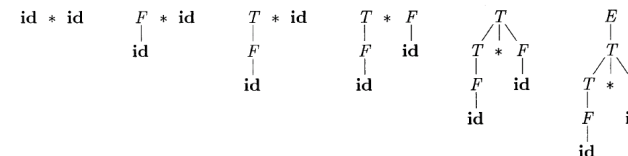
自底向上的语法分析

- 为一个输入串构造语法分析树的过程
- 从叶子（输入串中的终结符号，将位于分析树的底端）开始，向上到达根结点
- 重要的自底向上语法分析的**通用框架**
 - 移入-归约
- LR: 最大的可以构造出移入-归约语法分析器的语法类

自底向上的语法分析

- 自底向上的分析

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$



id * id 的自底向上分析过程

自底向上的语法分析

- 自底向上的分析：将一个串 w 归约为文法的开始符号过程
 - 在每一步的归约中，一个与某产生式体相匹配的特定子串被替换为该产生式头部的非终结符号，一次归约实质上是一个推导的反向操作
 - 核心操作：**移进 (shift)** 与 **归约 (reduce)**

自底向上的语法分析

- 自底向上的分析：将一个串 w 归约为文法的开始符号过程
 - 在每一步的归约中，一个与某产生式体相匹配的特定子串被替换为该产生式头部的非终结符号，**一次归约实质上是一个推导的反向操作（重要！）**

自底向上的语法分析

- 如何理解：一次归约实质上是一个推导的反向操作

自底向上的语法分析

- 如何理解：一次归约实质上是一个推导的反向操作
- 定义：设文法 $G=(V_N, V_T, P, S)$ ，单步推导记为 $\Rightarrow$
 - 若有产生式 $A \rightarrow \beta \in P$ ，则对任意上下文 α, ω ，我们有： $\alpha A \omega \Rightarrow \alpha \beta \omega$

在自底向上（移进-归约）分析中，单步归约恰好做相反的事：

- 若当前句型中存在某一子串与某条产生式右部 β 匹配（称为一个 **handle 句柄**，**注意，核心概念**），则

$$\alpha \beta \omega \xrightarrow{\text{reduce } A \rightarrow \beta} \alpha A \omega.$$

可见，归约（reduce）就是把“右部 β ”折回成“左部 A ”——这正是把一次推导 $A \Rightarrow \beta$ 反向执行

自底向上的语法分析

- 如何理解：一次归约实质上是一个推导的反向操作
- 自底向上分析的整条归约序列，等价于某个“最右推导（rightmost derivation）”的逆序
如果：

$$S = Y_0 \Rightarrow Y_1 \Rightarrow \dots \Rightarrow Y_n = w$$

是一条最右推导，那么自底向上的分析从 w 出发，按

$$\gamma_n \xrightarrow{\text{reduce}} \gamma_{n-1} \xrightarrow{\text{reduce}} \dots \xrightarrow{\text{reduce}} \gamma_0 = S$$

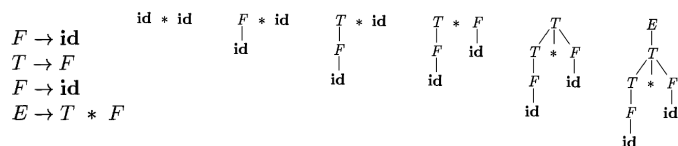
逐步把右部还原为左部，直到得到开始符号 S

自底向上的语法分析

- 自底向上的分析：将一个串 w 归约为文法的开始符号过程
 - 在每一步的归约中，一个与某产生式体相匹配的特定子串被替换为该产生式头部的非终结符号，一次归约实质上是一个推导的反向操作（重要！）
 - 核心操作：**移进（shift）**与**归约（reduce）**
 - 主要解决问题：
 - 什么时候应该移进
 - 什么时候应该归约

自底向上的语法分析

- $id * id$ 的归约过程：
 - $id * id, F * id, T * id, T * F, T, E$
- 对于句型 $T * id$ ，有两个子串和某产生式右部匹配
 - T 是 $E \rightarrow T$ 的右部
 - id 是 $F \rightarrow id$ 的右部
 - 为什么选择将 id 归约为 F ，而不是将 T 归约为 E ？
 - T 归约为 E 之后， $E * id$ 不再是句型



自底向上的语法分析

- “句柄（handle）”的作用
 - 定义：在一个右句型（right-sentential form） γ 中，句柄指这样的二元组 $(A \rightarrow \beta, \text{位置})$ ：用 β 在 γ 中的那次出现替换为 A 后，恰好得到该串在**最右推导**中的上一步句型
 - 因此，**选择句柄并归约**，恰好是把“最近的一步最右推导”逆向撤销
- 特别注意：
 - 要点1：自底向上并非“乱归约”，而是精确地**逆行一条最右推导**；“句柄识别”是关键
 - 要点2：把“右边位置最靠右”与“句柄一定出现在最右端”混淆。**句柄是在该右句型中“最近一步最右推导的产物”，其位置由推导历史决定，不必总在串的最右端**
 - 要点3：归约前后都是最右句型，无二义性的文法，其每个最右句型都有且只有一个句柄

自底向上的语法分析

- “句柄 (handle)” 的作用
 - 在归约时告诉我们 哪一部分可以折叠成一个非终结符
 - 一次归约 = 找到一个句柄并用产生式左部替换
- 对比
 - 推导 (自顶向下)：从起始符号一步步展开
 - 归约 (自底向上)：从输入串一步步缩回

自底向上的语法分析

- 问题：如何找到句柄？
- 关键：**句柄不是任意右部匹配，而是必须出现在规范推导的最右推导中**
 - 错误理解：凡是文法右部出现的地方都能归约
 - 正确理解：**只有对应最右推导路径上的子串才是句柄**

自底向上的语法分析

- 问题：如何找到句柄？
- 输入：id * id

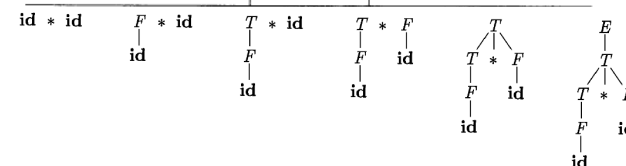
最右句型	句柄	归约用的产生式
id ₁ * id ₂		
F * id ₂		
T * id ₂		
T * F		

请思考，以上每个最右句型中，句柄是什么？

自底向上的语法分析

- 问题：如何找到句柄？
- 输入：id * id

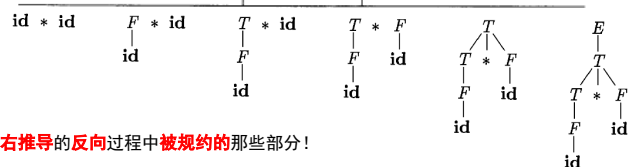
最右句型	句柄	归约用的产生式
id ₁ * id ₂	id ₁	$F \rightarrow id$
F * id ₂	F	$T \rightarrow F$
T * id ₂	id ₂	$F \rightarrow id$
T * F	T * F	$E \rightarrow T * F$



自底向上的语法分析

- 问题：如何找到句柄？
- 输入：id * id

最右句型	句柄	归约用的产生式
id ₁ * id ₂	id ₁	$F \rightarrow id$
F * id ₂	F	$T \rightarrow F$
T * id ₂	id ₂	$F \rightarrow id$
T * F	T * F	$E \rightarrow T * F$



句柄是**最右推导的反向过程**中被归约的那些部分！

自底向上的语法分析

- 移进归约语法分析框架：一种自底向上的分析形式
 - 使用一个栈保存文法符号，一个输入缓冲区存放将要进行分析的剩余符号
 - 初始栈：\$ 初始输入 w\$
 - 对输入串的一次从左到右的扫描过程中，语法分析器将零个或多个输入符号移到栈的顶端，直到它可以**对栈顶的一个文法符号串进行归约**为止
 - 它将归约为某个产生式的头。不断重复这个循环，直到它检测到一个语法错误，或者栈中包含了开始符号且输入缓冲区为空。
 - 分析成功时：栈 \$ S，输入 \$

自底向上的语法分析

- 移进归约语法分析框架：一种自底向上的分析形式
 - 使用一个栈来保存归约/扫描移入的文法符号
 - 栈中符号（从底向上）和待扫描的符号组成了一个最右句型
 - 开始时刻：栈中只包含\$，而输入为w\$
 - 成功结束时刻：栈中\$\$，而输入\$
 - 在分析过程中，不断地移入符号，并在识别到句型时进行归约
- 主要分析动作
 - 移入：将下一个输入符号移动到栈顶
 - 归约：将句柄归约为相应的非终结符号
 - 句柄总是在栈顶
 - 具体操作时弹出句柄，压入被归约到的非终结符号
 - 接受：宣布分析过程成功完成
 - 报错：发现语法错误，调用错误恢复子程序

自底向上的语法分析

- 问题来了：**冲突**
- 为什么会有冲突：
 - 移入-归约技术并不能处理所有上下文无关文法

自底向上的语法分析

- 还是看我们的“dangling else”示例：

```
S → if E then S
    | if E then S else S
    | other
```

输入串：if E then if E then other else other

```
S ⇒ if E then S
    ⇒ if E then if E then S else S
    ⇒ if E then if E then other else other
```

```
S ⇒ if E then S else S
    ⇒ if E then if E then S else S
    ⇒ if E then if E then other else other
```

我们可以得到两种最右推导过程，上方为else匹配外层，下方为else匹配里层

自底向上的语法分析

- 移进归约语法分析框架
- 图示例子：

栈	输入	动作
\$	id₁ * id₂ \$	移入
\$ id₁	* id₂ \$	按照 $F \rightarrow \text{id}$ 归约
\$ F	* id₂ \$	按照 $T \rightarrow F$ 归约
\$ T	* id₂ \$	移入
\$ T *	id₂ \$	移入
\$ T * id₂	\$	按照 $F \rightarrow \text{id}$ 归约
\$ T * F	\$	按照 $T \rightarrow T * F$ 归约
\$ T	\$	按照 $E \rightarrow T$ 归约
\$ E	\$	accept

输入 id*id 时的分析步骤

自底向上的语法分析

- 问题来了：**冲突**
- 为什么会有冲突：
 - 移入-归约技术并不能处理所有上下文无关文法
- 还是看我们的“dangling else”示例：

```
S → if E then S
    | if E then S else S
    | other
```

输入串：if E then if E then S else S

自底向上的语法分析

- 还是看我们的“dangling else”示例：

```
S → if E then S
    | if E then S else S
    | other
```

输入串：if E then if E then other else other

```
S ⇒ if E then S
    ⇒ if E then if E then S else S
    ⇒ if E then if E then other else other
```

```
S ⇒ if E then S else S
    ⇒ if E then if E then S else S
    ⇒ if E then if E then other else other
```

分析我们的输入串：if E then if E then other else other \$

我们可以得到两种最右推导过程，上方为else匹配外层，下方为else匹配里层

自底向上的语法分析

- 还是看我们的“dangling else”示例：

S → if E then S
| if E then S else S
| other

S ⇒ if E then S
⇒ if E then if E then S else S
⇒ if E then if E then other else other

输入串：if E then if E then other else other

分析我们的输入串：if E then if E then other else other \$

某一个时刻，栈推进到如下状态：
栈：if E then if E then other
输入：else other \$

S ⇒ if E then S else S
⇒ if E then if E then S else S
⇒ if E then if E then other else other

我们可以得到两种最右推导过程，上方为else匹配外层，下方为else匹配里层

冲突点出现！

自底向上的语法分析

- 还是看我们的“dangling else”示例：

S → if E then S
| if E then S else S
| other

此时栈顶匹配了 if E then S，可能有两种操作：

1.归约操作（Reduce）

- 句柄：if E then S
- 应用产生式：S → if E then S
- 结果：栈缩回为 if E then S，输入仍然是 else other \$

2.移进操作（Shift）

- 直接移进 else，形成更长的候选句柄
- 准备匹配产生式：S → if E then S else S

输入串：if E then if E then other else other

分析我们的输入串：if E then if E then other else other \$

某一个时刻，栈推进到如下状态：
栈：if E then if E then other
输入：else other \$

形成了冲突！
•在 **最右推导 1** 中，句柄确实是 if E then S → 应该归约
•在 **最右推导 2** 中，句柄是更长的 if E then S else S → 应该继续移进
同一个位置，可能存在两个不同的合法句柄
→ 这就是移进-归约冲突的本质。

自底向上的语法分析

- 规约-规约冲突

- 示例：

(1)	stmt	→	id (parameter_list)
(2)	stmt	→	expr := expr
(3)	parameter_list	→	parameter_list , parameter
(4)	parameter_list	→	parameter
(5)	parameter	→	id
(6)	expr	→	id (expr_list)
(7)	expr	→	id
(8)	expr_list	→	expr_list , expr
(9)	expr_list	→	expr



自底向上的语法分析

- 还是看我们的“dangling else”示例：

S → if E then S
| if E then S else S
| other

输入串：if E then if E then other else other

分析我们的输入串：if E then if E then other else other \$

某一个时刻，栈推进到如下状态：
栈：if E then if E then other
输入：else other \$

形成了冲突！
•在 **最右推导 1** 中，句柄确实是 if E then S → 应该归约
•在 **最右推导 2** 中，句柄是更长的 if E then S else S → 应该继续移进
同一个位置，可能存在两个不同的合法句柄
→ 这就是移进-归约冲突的本质。

此时栈顶匹配了 if E then S，可能有两种操作：

1.归约操作（Reduce）

- 句柄：if E then S
- 应用产生式：S → if E then S
- 结果：栈缩回为 if E then S，输入仍然是 else other \$

2.移进操作（Shift）

- 直接移进 else，形成更长的候选句柄
- 准备匹配产生式：S → if E then S else S

直观理解

- 归约：假设 if E then S 是完整语句 → else 另起作用
- 移进：认为 else 应该绑定到最近的 if

自底向上的语法分析

- 规约-规约冲突

E → E and E
E → E or E
E → id

- 示例：

➢输入：id and id or id

➢语义上我们希望：

- and 的优先级 高于 or
- 即应该解析为 (id and id) or id

➢但是在这个文法下，LR 分析器会遇到规约-规约冲突：

- 在读入 id and id 之后，分析栈已经能规约为一个 E
- 但同时，前两个 id 也能直接规约为 E → id（多种规约选择）
- 在下一个符号 or 到来时，分析器会发现：
 - 可以规约 E → E and E
 - 也可以规约 E → id（针对最后那个 id）
- 两个不同的 reduce 操作同时出现 ⇒ 规约-规约冲突

自底向上的语法分析

- 问题来了：**冲突**

- 为什么会有冲突：

- 文法允许多种推导方式（即 二义性文法）
- 语法分析器无法仅凭当前状态和下一个输入符号决定：
 - 现在就归约
 - 还是继续移进
 - 或者说，应该选择哪条产生式进行规约

- 两类冲突：

- 移进-规约冲突：分析器无法决定“移进”还是“规约”
- 规约-规约冲突：分析器无法决定“用哪条规则规约”

自底向上的语法分析

- 从输入串出发构造一个反向最右推导序列（归约）
- 每一步是对**句柄**进行归约 -- 寻找句柄
- 一种移进-归约的分析框架
 - 一个栈存放文法符号，一个输入缓冲区
 - 移进 or 归约
 - 总是能够在栈的顶端找到句柄
 - 通过证明是可行的
- 有时候会有冲突，是移进还是归约？ 怎么归约？

自底向上的语法分析

- 表格驱动
 - 虽然手工构造表格工作量大，但表格可以自动生成
- 对于几乎所有的程序设计语言，只要写出上下文无关文法，就能够构造出识别该构造的LR语法分析器
- 最通用的**无回溯移入-归约分析技术**，且和其它技术一样高效
- 可以尽早检测到错误
- LR(1) 可以识别所有确定性上下文无关文法
- 能分析的文法集合是LL(k)文法的超集

自底向上的语法分析

- LR(0)分析技术
- LR(0)项
 - 文法的一个产生式加上在其产生式体中某处的一个点
 - $A \rightarrow \cdot XYZ$, $A \rightarrow X \cdot YZ$, $A \rightarrow XY \cdot Z$, $A \rightarrow XYZ \cdot$
 - 注意: $A \rightarrow \epsilon$ 只对应一个项 $A \rightarrow \cdot$
 - 直观含义
 - 项 $A \rightarrow \alpha \cdot \beta$ 表示已经扫描/归约到了 α ，并期望接下来的输入中经过扫描/归约得到 β ，然后把 $\alpha\beta$ 归约到A
 - 如果 β 为空，表示我们可以把 α 归约为A
 - 项也可以用一对整数表示
 - (i,j) 表示第i条规则，点位于右部第j个位置

问题来了：上面的文法为例[1,2] 表示什么？

自底向上的语法分析

- LR语法分析技术
- LR(k)的语法分析概念
 - L表示最左扫描，R表示反向构造出最右推导
 - k表示最多向前看k个符号
- 当k的数量增大时，相应的语法分析器的规模急剧增大
 - K=2时，程序设计语言的语法分析器的规模通常非常庞大
 - 当k=0、1时已经可以解决很多语法分析问题，因此具有实践意义
 - 因此，我们只考虑k≤1的情况

自底向上的语法分析

- 移入-归约语法分析器如何知道何时该移入、何时该归约呢？
 - LR语法分析器试图用**一些状态**来表明我们在移进归约语法分析过程中所处的位置，从而做出移入-归约决定
- 项的意义
 - 指明在语法分析过程中的给定点上，我们**已经看到了一个产生式的哪些部分**。或者说，如果我们想用这个产生式进行归约，还需要看到哪些文法符号
- **项的集合（项集）对应于一个状态**

自底向上的语法分析

- LR(0)分析技术
- 规范LR(0)项集族
 - 规范LR(0)项集族提供构建LR(0)自动机的基础
 - LR(0)自动机可用于做出语法分析决定
 - LR(0)自动机中每个状态代表了LR(0)项集族中的一个项集

自底向上的语法分析

- LR(0)分析技术
- 以项为基础构造自动机（DFA）
 - 构造以项为状态的自动机
 - 开始状态 $S' \rightarrow \cdot S$
 - 转换
 - $A \rightarrow \alpha \cdot B \beta$ 到 $B \rightarrow \cdot \gamma$ 有一个 ϵ 转换
 - 从 $A \rightarrow \alpha \cdot X \beta$ 到 $A \rightarrow \alpha X \cdot \beta$ 有一个 X 转换
 - 接受状态: $A \rightarrow \alpha \cdot$, 即点在最后的项

自底向上的语法分析

- LR(0)分析技术
- 规范LR(0)项集族的构造
- 增广文法
 - G的增广文法G'是在G中增加新开始符号S', 并加入产生式 $S' \rightarrow S$ 而得到的
 - G'和G接受相同的语言, 且按照 $S' \rightarrow S$ 进行归约实际上就表示已经将输入符号串归约成为开始符号
 - 引入的目的是告诉语法分析器何时宣布接受输入符号串, 即用 $S' \rightarrow S$ 进行归约时, 表明分析结束

自底向上的语法分析

- CLOSURE(I)的构造算法
- 如果I是文法G的一个项集, 那么CLOSURE(I)就是根据下列规则从I构造得到的项集
 - 将I中的各个项加入到CLOSURE(I)中
 - 如果 $A \rightarrow \alpha \cdot B \beta$ 在CLOSURE(I)中, 那么对B的任意产生式 $B \rightarrow \gamma$, 将 $B \rightarrow \cdot \gamma$ 加入到CLOSURE(I)中
 - 不断重复第二步, 直到收敛
- 第二步的意义
 - 项 $A \rightarrow \alpha \cdot B \beta$ 表示期望在接下来的输入中归约到B
 - 显然, 要归约到B, 首先要扫描归约到B的某个产生式的右部
 - 因此对每个产生式 $B \rightarrow \gamma$, 加入 $B \rightarrow \cdot \gamma$
 - 表示它期望能够扫描归约到 γ

自底向上的语法分析

- LR(0)分析技术
- 规范LR(0)项集族的构造
- 三个概念
 - 增广文法
 - 项集闭包: CLOSURE
 - GOTO

自底向上的语法分析

- LR(0)分析技术
- 规范LR(0)项集族的构造
 - CLOSURE(I): I的项集闭包
 - 对应于DFA化算法的 ϵ -CLOSURE
 - GOTO(I,X): I的X后继
 - 对应于DFA化算法的MOVE(I,X)

自底向上的语法分析

- 增广文法:
 - $E' \rightarrow E \quad E \rightarrow E+T \mid T \quad T \rightarrow T * F \mid F \quad F \rightarrow (E) \mid id$
- 项集 $I = \{[E' \rightarrow \cdot E]\}$ 的闭包
 - $[E' \rightarrow \cdot E]$ 在闭包中
 - $[E \rightarrow \cdot E+T], [E \rightarrow \cdot T]$ 在闭包中
 - $[T \rightarrow \cdot T * F], [T \rightarrow \cdot F]$ 在闭包中
 - $[F \rightarrow \cdot (E)], [F \rightarrow \cdot id]$ 在闭包中

```

SetOfItems CLOSURE(I) {
    J = I;
    repeat
        for (J 中的每个项  $A \rightarrow \alpha \cdot B \beta$ )
            for (G 的每个产生式  $B \rightarrow \gamma$ )
                if (项  $B \rightarrow \cdot \gamma$  不在 J 中)
                    将  $B \rightarrow \cdot \gamma$  加入 J 中;
    until 在某一轮中没有新的项被加入到 J 中;
    return J;
}

```

自底向上的语法分析

- LR(0)项集中的内核项和非内核项
 - 内核项：初始项 $S' \rightarrow \cdot S$ 、以及所有点不在最左边的项
 - 非内核项：除了 $S' \rightarrow \cdot S$ 之外、点在最左边的项
- 由于表可能很庞大，实现算法时可以考虑只保存相应的非终结符号；甚至可以只保存内核项，而在要使用非内核项时调用 CLOSURE 函数重新计算

自底向上的语法分析

- 规范LR(0)项集族的构造
- 为文法G构造LR(0)项集规范族C
 - 步骤一：构造增广文法G': $S' \rightarrow S$
 - 步骤二：构造 $I_0 = \text{CLOSURE}(S' \rightarrow \cdot S)$ ， I_0 是C的初始项集
 - 步骤三：对C中的每个项集 I_i 及每个文法符号 X_j ，将 $\text{GOTO}(I_i, X_j)$ 加入到C中
 - 重复步骤三，直到没有新的项集可以加入

```
void items(G') {
    C = CLOSURE({S' → · S});
    repeat
        for (C中的每个项集 I)
            for (每个文法符号 X)
                if (GOTO(I, X) 非空且不在 C中)
                    将 GOTO(I, X) 加入 C中;
    until 在某一轮中没有新的项集被加入到 C 中;
}
```

自底向上的语法分析

- 规范LR(0)项集族的构造示例：
 - 初始 $I_0 = \text{closure}(\{E' \rightarrow \cdot E\})$ 会包含下列项目：
 - $E' \rightarrow \cdot E$
 - $E \rightarrow \cdot E + T$
 - $E \rightarrow \cdot T$
 - $T \rightarrow \cdot T * F$
 - $T \rightarrow \cdot F$
 - $F \rightarrow \cdot (E)$
 - $F \rightarrow \cdot \text{id}$

说明：因为 $\cdot$ 前面是 E（非终结符），我们把 E 的产生式也加入；在 $E \rightarrow \cdot T$ 中 $\cdot$ 前是 T，我们继续把 T 的产生式加入；同理把 F 的产生式加入，直到 closure 完成

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow \text{id}$

自底向上的语法分析

- GOTO函数
- I 是一个项集，X 是一个文法符号， $\text{GOTO}(I, X)$ 定义为 I 中所有形如的项 $[A \rightarrow \alpha \cdot X \beta]$ 所对应的项 $[A \rightarrow \alpha X \cdot \beta]$ 的集合的闭包
 - 根据项的历史-期望的含义， $\text{GOTO}(I, X)$ 表示读取输入中的 X 或者归约到一个 X 之后的情况
 - $\text{GOTO}(I, X)$ 定义了 LR(0) 自动机中状态 I 在 X 之上的转换
- 例如：
 - $I = \{[E' \rightarrow E \cdot], [E \rightarrow E \cdot + T]\}$
 - $\text{GOTO}(I, +)$ 计算如下
 - I 中只有一个项的点后面跟着 +，对应的项为 $[E \rightarrow E + \cdot T]$
 - $\text{CLOSURE}(\{[E \rightarrow E + \cdot T]\}) = \{[E \rightarrow E + \cdot T], [T \rightarrow \cdot T * F], [T \rightarrow \cdot F], [F \rightarrow \cdot (E)], [F \rightarrow \cdot \text{id}]\}$

自底向上的语法分析

- 规范LR(0)项集族的构造
- 为文法G构造LR(0)项集规范族C的关键步骤总结
 - LR(0) 项目：形如 $A \rightarrow \alpha \cdot \beta$ ，点 ($\cdot$) 表示在该产生式“已经识别到”的位置
 - $\text{closure}(I)$ ：若 $A \rightarrow \alpha \cdot B \beta \in I$ ($\cdot$ 在非终结符 B 前)，则把所有 $B \rightarrow \gamma$ 加入 I，并重复直到不再能加入
 - $\text{goto}(I, X)$ ：对集合 I 中所有 $A \rightarrow \alpha \cdot X \beta$ ，把点向右移一位得 $A \rightarrow \alpha X \cdot \beta$ ，然后对该新集合做 closure，得到 $\text{goto}(I, X)$
 - 从初始 $I_0 = \text{closure}(\{E' \rightarrow \cdot E\})$ 出发，反复计算 goto，直到无新集合为止 → 得到项目集族 (states)

给出来一个 step by step 的例子?

自底向上的语法分析

- 规范LR(0)项集族的构造示例：
 - $\text{goto}(I_0, \text{id})$ ：由 $F \rightarrow \cdot \text{id}$ 得 $F \rightarrow \text{id} \cdot$ （这是一个完成项，表示能规约 $F \rightarrow \text{id}$ ）；closure 不再添加新项
 - $\text{goto}(I_0, '(')$ ：由 $F \rightarrow \cdot (E)$ 得 $F \rightarrow (\cdot E)$ ，此时 $\cdot$ 前是非终结符 E，因此需要把 $E \rightarrow \cdot E + T$, $E \rightarrow \cdot T$ 等产生式一并加入，这会形成一个结构近似于 I_0 的子集（但 $\cdot$ 在不同位置）
 - $\text{goto}(I_0, E)$ ：由 $E' \rightarrow \cdot E$ 与 $E \rightarrow \cdot E + T$ 等项目得到 $E' \rightarrow E \cdot$ 与 $E \rightarrow E \cdot + T$ ，这构成了一个状态（可以记为 I_1 ），表示“已识别一个完整的 E，但也可能看到 + 接着更多的东西”
 - $\text{goto}(I_0, T)$, $\text{goto}(I_0, F)$

自底向上的语法分析

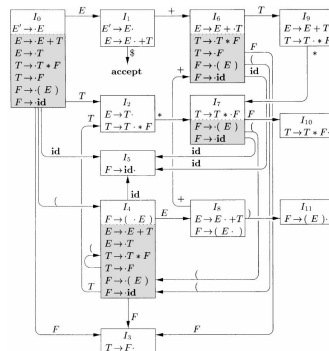
1. 初始化：栈 = \$ 0, 输入 = 字符串 + \$
2. 查表：
 - 看栈顶状态 = i, 当前输入符号 = a
 - 查表项：ACTION[i, a]
3. 执行动作：
 - 移进 S_j:
 - ❑ 把 a 和状态 j 压入栈
 - ❑ 输入指针前移
 - 规约 R A → α:
 - ❑ 从栈中弹出 2 × |α| 个元素（符号和状态）
 - ❑ 栈顶状态 = i, 查 GOTO[i, A] = j
 - ❑ 压入 A_j
 - 接受 ACC:
 - ❑ 输入串符合文法，分析成功
 - 出错：
 - ❑ ACTION 表为空，输入非法
4. 重复：直到 ACC 或出错

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

• 举例说明：id * id \$

行号	栈	符号	输入	动作
(1)	0	\$	id * id \$	移到 5
(2)	0 5	\$ id	* id \$	按照 $F \rightarrow id$ 归约
(3)	0 3	\$ F	* id \$	按照 $T \rightarrow F$ 归约
(4)	0 2	\$ T	* id \$	移到 7
(5)	0 2 7	\$ T *	id \$	移到 5
(6)	0 2 7 5	\$ T * id	\$	按照 $F \rightarrow id$ 归约
(7)	0 2 7 10	\$ T * F	\$	按照 $T \rightarrow T * F$ 归约
(8)	0 2	\$ T	\$	按照 $E \rightarrow T$ 归约
(9)	0 1	\$ E	\$	接受

自底向上的语法分析



sX 表示 shift 到状态 X；
rk 表示 reduce 用产生式 k（例如 r6 表示用 (6) $F \rightarrow id$ 进行规约）；

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

• 举例说明：id * id \$

行号	栈	符号	输入	动作
(1)	0	\$	id * id \$	移到 5
(2)	0 5	\$ id	* id \$	按照 $F \rightarrow id$ 归约
(3)	0 3	\$ F	* id \$	按照 $T \rightarrow F$ 归约
(4)	0 2	\$ T	* id \$	移到 7
(5)	0 2 7	\$ T *	id \$	移到 5
(6)	0 2 7 5	\$ T * id	\$	按照 $F \rightarrow id$ 归约
(7)	0 2 7 10	\$ T * F	\$	按照 $T \rightarrow T * F$ 归约
(8)	0 2	\$ T	\$	按照 $E \rightarrow T$ 归约
(9)	0 1	\$ E	\$	接受

自底向上的语法分析

- 假设文法符号串 γ 使 LR(0) 自动机从开始状态运行到状态（项集）j
 - 如果 j 中有一个形如 $A \rightarrow \alpha$ 的项，那么
 - ❑ 在 γ 之后添加一些终结符号可以得到一个最右句型
 - ❑ α 是 γ 的后缀，且 $A \rightarrow \alpha$ 是这个句型的句柄
 - ❑ 表示可能找到了当前最右句型的句柄
 - 如果 j 中存在一个项 $B \rightarrow \alpha.X\beta$ ，那么
 - ❑ 在 γ 之后添加 $X\beta$ ，然后再添加一个终结符号串可得到一个最右句型
 - ❑ 在这个句型中 $B \rightarrow \alpha.X\beta$ 是句柄
 - ❑ 此时表示还没有找到句柄，需要移入

自底向上的语法分析

- 一个关键的问题？！
- LR(0) 在做“规约”决策时不看任何前瞻符号
 - 也就是说一旦某个项目集里出现了完成项 $A \rightarrow \alpha$ ，LR(0) 会在所有可能的输入符号上把它当作可规约动作（除非该状态上还有移进动作，产生移进-规约冲突）

自底向上的语法分析

- LR(0) 自动机的使用
 - 移入-归约时，LR(0) 自动机被用于识别句型
 - 已经归约/移入得到的文法符号序列对应于 LR(0) 自动机的一条路径
 - 如果路径到达接受状态，表明栈上端的某个符号串可能是句柄
- 不需要每次用归约/移入得到的串来运行 LR(0) 自动机
 - 路径被放到栈中；且文法符号可以省略，因为由 LR(0) 状态可以确定相应文法符号
 - 在移入后，根据原来的栈顶状态即可知道新的状态
 - 在归约时，根据归约产生式的右部长度弹出相应状态，仍然可以根据此时的栈顶状态计算得到新状态

自底向上的语法分析

- 一个关键的问题？！
- LR(0) 在做“规约”决策时不看任何前瞻符号
 - 也就是说一旦某个项目集里出现了完成项 $A \rightarrow \alpha$ ，LR(0) 会在所有可能的输入符号上把它当作可规约动作（除非该状态上还有移进动作，产生移进-规约冲突）
- 对于这个表达式文法，许多状态会同时包含：
 - 某个完成项（例如 $F \rightarrow id$ 或 $T \rightarrow F$ 等），以及
 - 另一个项目如 $T \rightarrow T * F$ 或 $E \rightarrow E + T$ （点在非终结符或终结符前）
- 因此，当分析器遇到某些输入（例如 id + id * id 的中间位置），会产生移进-规约冲突或规约-规约冲突

LR(0) 无法凭借其有限状态做出正确、唯一的决策

自底向上的语法分析

- 讨论：LR(0) 的局限性
 - 有些文法会产生 移进-规约冲突 或 规约-规约冲突
 - LR(0) 过于严格，能处理的文法较少
 - 根本原因：
 - LR(0) 只看项目中的“点位置”，并不知道后面真正可能出现的符号
 - 因此在“能规约”和“能移进”的情形下，它无法做出唯一选择
- 改进方案是什么？
 - 更强的算法（SLR, LR(1), LALR）就是通过前瞻或 FOLLOW 来减少冲突

自底向上的语法分析

- SLR: Simple LR 算法的核心
 - 基本思想：
 - 在 LR(0) 项目集基础上，加上 FOLLOW 集来控制规约动作
 - 规约 $A \rightarrow \alpha$ 只在 输入符号 $\in \text{FOLLOW}(A)$ 时才执行
 - 优点：
 - 消除了大部分 LR(0) 的冲突
 - 构造过程和 LR(0) 类似，只需在生成 ACTION 表时加上 FOLLOW 集限制
 - 缺点：
 - 仍然可能存在不足（FOLLOW 集过大，可能引入不必要的冲突）

自底向上的语法分析

- SLR: Simple LR 算法的核心
 - 如果不仅考虑First，而且把Follow集也考虑在内，就可以一定程度上解决 reduce-shift conflict. 还是以上语法：
 - $F \rightarrow Y \cdot + B$
 - $F \rightarrow Y \cdot$
 - 如果 $\text{FOLLOW}(F) = \{a, b\}$ ，那么当我们遇到a与b时，就可以选择归约F；当我们遇到+时，就可以选择移进操作

自底向上的语法分析

- 回顾：
 - First函数：由某个非终结符能够推导出来的句子的句首符号的集合，就是这个非终结符的First集合
 - Follow函数：对于某个非终结符，紧跟在其后面的终结符的集合就是这个非终结符的Follow集

自底向上的语法分析

- SLR: Simple LR 算法的核心
- Follow集的计算：

自底向上的语法分析

- 输入：一个增广文法 G'
- 输出： G' 的SLR语法分析表函数ACTION和GOTO
- 方法：
 - 1) 构造 G' LR(0)项集规范族 $C = \{I_0, I_1, \dots, I_n\}$
 - 2) 对于状态 i 中的项：
 - $[A \rightarrow \alpha a \beta]$ 且 $\text{GOTO}[I_i, a] = I_j$, 那么 $\text{ACTION}[i, a] = \text{移入 } j$
 - $[A \rightarrow \alpha \cdot]$, 那么对于 $\text{FOLLOW}(A)$ 中的所有 a , $\text{ACTION}[i, a] = \text{归约 } A \rightarrow \alpha$
 - $[S' \rightarrow S \cdot]$, 那么 $\text{ACTION}[i, \$] = \text{接受}$
 - 3) 状态 i 对于非终结符号 A 的转换规则：
 - 如果 $\text{GOTO}[I_i, A] = I_j$, 那么 $\text{GOTO}[i, A] = j$
 - 4) 规则2)和3)没有定义的所有条目设置为“报错”

自底向上的语法分析

• SLR: Simple LR 算法的核心

- (0) $E' \rightarrow E$
- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

FOLLOW 集结果:

$FOLLOW(E) = \{ \$, + \}$

说明: 来自 $E' \rightarrow E$ 得到 $\$$; $F \rightarrow (E)$ 得到 $\$$; $E \rightarrow E + T$ 在 $+$ 后面有 T , 与 $FOLLOW(E)$ 无关; 但在 $E \rightarrow E + T$ 中, E 后面直接是 $+$, 因此 $+$ $\in FOLLOW(E)$.

$FOLLOW(T) = \{ \$, + \}$

说明: $E \rightarrow E + T$ 给 $FOLLOW(T)$ 包含 $FOLLOW(E)$; 且 T 在 $E \rightarrow E + T$ 末尾, $FOLLOW(T) \supseteq FOLLOW(E)$; 综合为 $\{ \$, + \}$.

$FOLLOW(F) = \{ \$, +, * \}$

说明: $T \rightarrow T * F$ 给 $*$ 不影响 $FOLLOW(F)$ (F 前面是 $*$), 但 F 在式尾, $FOLLOW(F) \supseteq FOLLOW(T)$; 同时在 $T \rightarrow T * F$ 中, F 后无符号, 故 $FOLLOW(F) \supseteq FOLLOW(T)$; 最终 $\{ \$, +, * \}$.

$FOLLOW(E) = \{ +,), \$ \}$
 $FOLLOW(T) = \{ +, *,), \$ \}$
 $FOLLOW(F) = \{ +, *,), \$ \}$

在某状态出现 $[E \rightarrow T]$

在 **LR(0)**: 会在所有终结符上放 "reduce $E \rightarrow T$ "

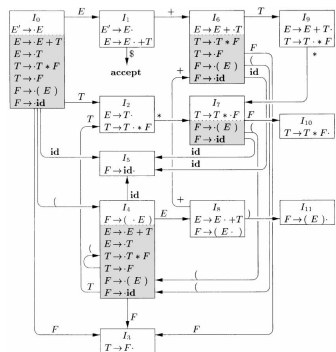
• 所以在 $*$ 上冲突 (因为 $[T \rightarrow T * F]$ 允许移进)

在 **SLR**: 只在 $FOLLOW(E) = \{ +,), \$ \}$ 上放规约

• 在 $*$ 上不会规约 $\rightarrow$ 冲突消除了

自底向上的语法分析

• SLR: Simple LR 算法的核心



- (0) $E' \rightarrow E$
- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

$FOLLOW(E) = \{ +,), \$ \}$
 $FOLLOW(T) = \{ +, *,), \$ \}$
 $FOLLOW(F) = \{ +, *,), \$ \}$

状态	id	(	)	+	*	\$	E	T	F
0	s5	s4					1	2	3
1				s6		acc			
2			r2	r2	s7	r2			
3			r4	r4	r4	r4			
4	s5	s4					8	2	3
5			r6	r6	r6	r6			
6	s5	s4					9	3	
7	s5	s4						10	
8			s11	s6					
9			r1	r1	s7	r1			
10			r3	r3	r3	r3			
11			r5	r5	r5	r5			

自底向上的语法分析

• SLR语法分析器的弱点

➢ SLR不能完全解决reduce-shift conflict.

➢ 同样还是上面的语法:

□ $F \rightarrow Y \cdot + B$

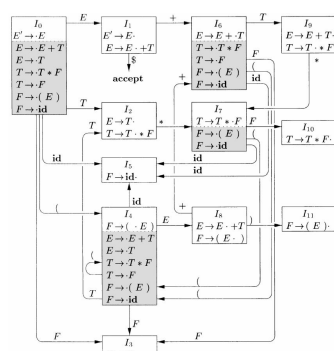
□ $F \rightarrow Y \cdot$

➢ 如果 $FOLLOW(F) = \{ a, b, + \}$, 那我们遇到 $+$ 符号时, 就无法确定到底是选择移进操作得到 $F \rightarrow Y \cdot + B$, 还是归约 F .

➢ SLR不能完全解决reduce-shift conflict. 这就是为什么我们要选择LR(1) / LALR(1)了

自底向上的语法分析

• SLR: Simple LR 算法的核心



- (0) $E' \rightarrow E$
- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

$FOLLOW(E) = \{ +,), \$ \}$
 $FOLLOW(T) = \{ +, *,), \$ \}$
 $FOLLOW(F) = \{ +, *,), \$ \}$

状态	id	(	)	+	*	\$	E	T	F
0	s5	s4					1	2	3
1				s6		acc			
2	r2	r2	r2	r2	s7/r2	r2			
3	r4	r4	r4	r4	r4	r4			
4	s5	s4					8	2	3
5	r6	r6	r6	r6	r6	r6			
6	s5	s4						9	3
7	s5	s4							10
8			s11	s6					
9	r1	r1	r1	r1	s7/r1	r1			
10	r3	r3	r3	r3	r3	r3			
11	r5	r5	r5	r5	r5	r5			

自底向上的语法分析

• 根据上一算法构造的语法分析表, 若表中各位置没有多个条目, 则称为文法G的SLR(1)分析表。使用该分析表的分析器, 称为G的SLR(1)语法分析器。G称为SLR(1)文法

• 若表中某位置存在多个条目 (多个可选的动作, 冲突), 则该文法是非SLR(1)文法

总结: SLR(1) = 在 LR(0) 基础上 + FOLLOW 过滤

→ 简单高效, 但能力有限; LR(1) 更强, 但代价是状态数暴增。

自底向上的语法分析

• SLR技术解决冲突的方法

➢ 项集中包含 $[A \rightarrow \alpha \cdot]$ 时, 按照 $A \rightarrow \alpha$ 进行归约的条件是下一个输入符号 a 可以在某个句型中跟在 A 之后

□ 如果对于 a 还有其它的移入/归约操作, 则出现冲突, 参考如下文法:

$S' \rightarrow S \quad S \rightarrow L = R \quad S \rightarrow R \quad L \rightarrow * R \quad L \rightarrow id \quad R \rightarrow L$

➢ SLR无法解决的冲突来源:

□ 我们不妨假设, 项 $R \rightarrow L \cdot$ 出现在某个状态 X 中

□ 同时在 X 中还会因为 L 的后继可能是 $" = "$ 而存在对 $" = "$ 的移入 (请回忆项集闭包的算法, 来自 $L \rightarrow id \cdot$ 或 $L \rightarrow * R \cdot$ 后若面临 $" = "$)

□ $FOLLOW(R)$ 通常包含 $" = "$ (因为 $S \rightarrow L = R$ 且 R 在末尾, $FOLLOW(R) \supseteq FOLLOW(S)$; S 的 $FOLLOW$ 里至少有 $\$$, 并在项目集传播后 $" = "$ 也会进入 $FOLLOW(R)$ 的可触达位置)

□ 结果: 在该状态的 $" = "$ 列同时出现移入与规约 $r(R \rightarrow L)$, 导致 SLR 移入-规约冲突 (请注意, 实际上, 按照我们匹配最长的原则, $" = "$ 这个符号在某些上下文中意味着 "应该移入, 继续匹配 $L = R"$, 而不是立刻将 L 规约成 R 。)

自底向上的语法分析

• SLR技术解决冲突的方法

- 项集中包含 $[A \rightarrow \alpha.]$ 时，按照 $A \rightarrow \alpha$ 进行归约的条件是下一个输入符号a可以在某个句型中跟在A之后

□如果对于a还有其它的移入/归约操作，则出现冲突，参考如下文法：

$S' \rightarrow S \quad S \rightarrow L = R \quad S \rightarrow R \quad L \rightarrow * R \quad L \rightarrow id \quad R \rightarrow L$

- SLR无法解决的冲突来源：

- 我们不妨假设，项 $R \rightarrow L \cdot$ 出现在某个状态X中
- 同时在X中还会因为L的后继可能是“=”而存在对“=”的移入（请回忆项集闭包的算法，来自 $L \rightarrow id \cdot$ 或 $L \rightarrow *R \cdot$ 后若面临“=”）
- FOLLOW(R)通常包含“=”（因为 $S \rightarrow L = R$ 且R在末尾， $FOLLOW(R) \supseteq FOLLOW(S)$ ；S的FOLLOW里至少有\$，并在项目集传播后“=”也会进入FOLLOW(R)的可触达位置）
- 结果：**在该状态的“=”列同时出现移入与规约 $r(R \rightarrow L)$ ，导致SLR移入-规约冲突（请注意，实际上，按照我们匹配最长的原则，“=”这个符号在某些上下文中意味着“应该移入，继续匹配 $L = R$ ”，而不是立刻将L规约成R。）**

它对完成项使用FOLLOW(A)里所有符号进行规约，但FOLLOW(A)是全局的、与上下文无关的集合；

自底向上的语法分析

• LR(1)分析技术：

- LR(1)项的定义：形如 $[A \rightarrow \alpha \cdot \beta, a]$ ，其中a是展望符（lookahead），表示“当且仅当接下来看到a时，才考虑对 $A \rightarrow \alpha \cdot$ 做规约或其他动作”；

自底向上的语法分析

• LR(1)分析技术：

- LR(1)项的定义：形如 $[A \rightarrow \alpha \cdot \beta, a]$ ，其中a是展望符（lookahead），表示“当且仅当接下来看到a时，才考虑对 $A \rightarrow \alpha \cdot$ 做规约或其他动作”；

• LR(1)的核心区别

- 对完成项 $[A \rightarrow \alpha \cdot, a]$ ，仅在输入符号是a时才规约；
- 不再使用全局的FOLLOW(A)；而是每个项目携带精确到上下文的展望符集合。

自底向上的语法分析

- 规范LR方法。充分利用向前看符号。这种方法是用了更大的项集，称为LR(1)项集

- 向前看LR，又称LALR = Lookahead LR

自底向上的语法分析

• LR(1)分析技术：

- LR(1)项的定义：形如 $[A \rightarrow \alpha \cdot \beta, a]$ ，其中a是展望符（lookahead），表示“当且仅当接下来看到a时，才考虑对 $A \rightarrow \alpha \cdot$ 做规约或其他动作”；

• 在状态中包含更多的信息

- 让LR分析器的每个状态精确指明哪些输入符号可以跟在句柄 α 后面，使 α 可能被规约为A

• 让项包含第二个分量

- 形式如 $[A \rightarrow \alpha \cdot \beta, a]$ ，其中 $A \rightarrow \alpha \beta$ 是一条产生式，a是一个终结符号或者\$符
- a是向前看符号，长度为1

自底向上的语法分析

• LR(1)分析技术：

• 项 $[A \rightarrow \alpha \cdot \beta, a]$

- 如果 $\beta \neq \epsilon$ 时，a没有任何作用
- a的意义是若栈顶状态中存在LR(1)项 $[A \rightarrow \alpha \cdot, a]$ ，在输入符号为a时才会进行归约
- a的集合总是FOLLOW(A)的子集**

- **a指出能够进行归约的准确判断**

自底向上的语法分析

- LR(1)分析技术原理分析（概要，重要）：
 - 在某个状态 X 中（由 id 或 $*R$ 推进而来）会含有：
 - $[L \rightarrow id \cdot, =]$ 或 $[L \rightarrow *R \cdot, =]$ ，这意味着“当下一个符号是 $=$ 时，应继续匹配赋值，因此对 $=$ 执行移入”
 - 同时，状态 X 也可能包含：
 - $[R \rightarrow L \cdot, a]$ ，但其展望符 a 来自“ R 后面在上层文法中能看到的符号”。对于赋值产生式 $S \rightarrow L = R$ ， R 的展望来自 FOLLOW(S)（如 $\$$ ）或来自上文的 FIRST 传播，而“ $=$ ”并不是在 R 后出现的展望符
 - 因此，在 X 的“ $=$ ”列：
 - LR(1)：只看到移入（因为没有任何完成项以展望符“ $=$ ”要求规约）
 - SLR：因为 FOLLOW(R) 过宽，会把 $r(R \rightarrow L)$ 也放到“ $=$ ”列，造成 s/r 冲突

自底向上的语法分析

- LR(1)项集的构造
- 过程类似于LR(0)项集构造
- 多了一个向前看符号分量的计算
- CLOSURE
- GOTO

自底向上的语法分析

- LR(1)项集的构造的闭包CLOSURE算法

```

SetOfItems CLOSURE( $I$ ) {
    repeat
        for ( $I$  中的每个项  $[A \rightarrow \alpha \cdot B \beta, a]$ )
            for ( $G'$  中的每个产生式  $B \rightarrow \gamma$ )
                for (FIRST( $\beta a$ )中的每个终结符号  $b$ )
                    将  $[B \rightarrow \cdot \gamma, b]$  加入到集合  $I$  中;
    until 不能向  $I$  中加入更多的项;
    return  $I$ ;
}

```

自底向上的语法分析

- 规范LR(1)项
- 从可行前缀和有效项的角度看
- LR(1)项 $[A \rightarrow \alpha \cdot \beta, a]$ 对于一个可行前缀 $\delta \alpha$ 有效的条件是存在一个推导 $S \xRightarrow{*}_{rm} \delta A w \xRightarrow{*}_{rm} \delta \alpha \beta w$ ，其中 $a \in \text{First}(w)$ 。当 w 为 ϵ ， $a = \$$

$$S' \rightarrow S \quad S \rightarrow L = R \quad S \rightarrow R \quad L \rightarrow *R \quad L \rightarrow id \quad R \rightarrow L$$

自底向上的语法分析

- LR(1)项集的构造
- $\text{CLOSURE}(I) = I \cup \{[B \rightarrow \cdot \eta, b] \mid [A \rightarrow \alpha \cdot B \beta, a] \in \text{CLOSURE}(I), b \in \text{First}(\beta a), \text{ 且 } B \rightarrow \eta \text{ 为文法规则}\}$
 - 为什么 b 必须在 $\text{First}(\beta a)$ 中？
 - 对于可行前缀 γ 的有效项 $[A \rightarrow \alpha \cdot B \beta, a]$ ，存在最右推导 $S \xRightarrow{*}_{rm} \delta A a x \xRightarrow{*}_{rm} \delta \alpha B \beta a x$ ，其中 $\gamma = \delta \alpha$
 - 假设 $\beta a x$ 推导出终结符号串 $b \gamma$ ，那么对于某个形如 $B \rightarrow \eta$ 的产生式，有推导 $S \xRightarrow{*}_{rm} \gamma B b y \xRightarrow{*}_{rm} \gamma \eta b y$ ，因此， $\{[B \rightarrow \cdot \eta, b] \text{ 是 } \gamma \text{ 的有效项}\}$
 - b 是 $\text{First}(\beta a)$ 中的终结符号
 - b 可能是从 β 推导出的第一个终结符号
 - 也可能 β 推导出了 ϵ ， b 就是 a

自底向上的语法分析

- LR(1)项集的构造的GOTO函数算法

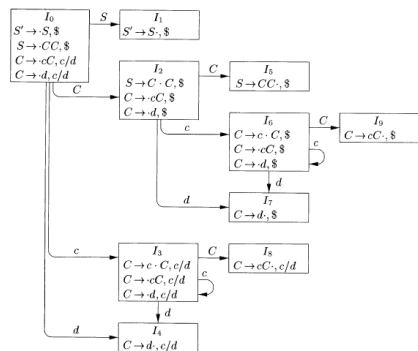
```

SetOfItems GOTO( $I, X$ ) {
    将  $J$  初始化为空集;
    for ( $I$  中的每个项  $[A \rightarrow \alpha \cdot X \beta, a]$ )
        将项  $[A \rightarrow \alpha X \cdot \beta, a]$  加入到集合  $J$  中;
    return CLOSURE( $J$ );
}

```

自底向上的语法分析

• LR(1)项集的构造示例



$S' \rightarrow S$
 $S \rightarrow C C$
 $C \rightarrow c C \mid d$

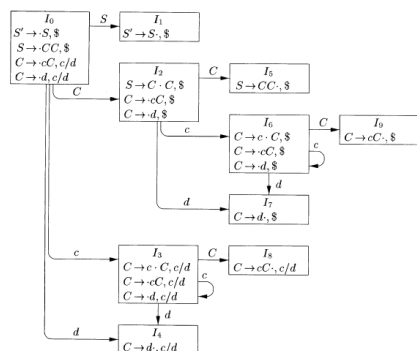
自底向上的语法分析

• LR(1)项集的构造分析表

- 输入：一个增广文法G'
- 输出：G'的规范LR语法分析表函数ACTION和GOTO
- 方法：
 - 1) 构造G' LR(1)项集规范族C={I0, I1, ..., In}
 - 2) 对于状态i中的项：
 - [A → α·aβ, b]且GOTO[i, a]=lj, 那么ACTION[i, a]=移入j
 - [A → α·, a], 且A ≠ S', ACTION[i, a]=归约A → α
 - [S' → S·, \$], 那么ACTION[i, \$]=接受
 - 3) 状态i对于非终结符号A的转换规则：如果
 - GOTO[i, A]=lj, 那么GOTO[i, A]=j
 - 4) 规则2)和3)没有定义的所有条目设置为“报错”

自底向上的语法分析

• LR(1)项集的构造示例

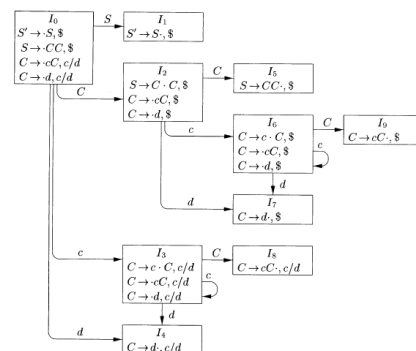


$S' \rightarrow S$
 $S \rightarrow C C$
 $C \rightarrow c C \mid d$

状态	c	d	\$	S	C
0					
1					
2					
3					
4					
5					
6					
7					
8					
9					

自底向上的语法分析

• LR(1)项集的构造示例



$S' \rightarrow S$
 $S \rightarrow C C$
 $C \rightarrow c C \mid d$

状态	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

自底向上的语法分析

• LR(1)项集的构造示例

• 注意到：

- 在(3,6) (4,7) (8,9)中，项的第一个分量是一样的，实质上它们来自于同样的LR(0)状态

$S' \rightarrow S$
 $S \rightarrow C C$
 $C \rightarrow c C \mid d$

状态	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

自底向上的语法分析

• LR(1)语法分析器的状态过多

• 但LR(1)的分析能力强于SLR(1)

• LALR分析

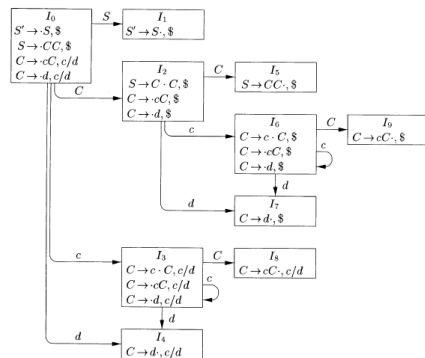
- 状态和SLR一样多
- 能力又比SLR稍微强一些

• 项集的合并！

- 将LR(1)项集中具有相同核心的项集合并为一个项集
- 项集核心是指LR(1)项集中第一个分量的集合
- 被合并项集的GOTO目标显然也可以被合并

自底向上的语法分析

- $I_3 = \{ [C \rightarrow c \cdot C, c/d], [C \rightarrow \cdot cC, c/d], [C \rightarrow \cdot d, c/d] \}$
- $I_6 = \{ [C \rightarrow c \cdot C, \$], [C \rightarrow \cdot cC, \$], [C \rightarrow \cdot d, \$] \}$
- 合并后:
 ➤ $I_{36} = \{ [C \rightarrow c \cdot C, c/d/\$], [C \rightarrow \cdot cC, c/d/\$], [C \rightarrow \cdot d, c/d/\$] \}$
- GOTO(I_3, C)和GOTO(I_6, C)显然也可以合并.....



自底向上的语法分析

- 合并引起的冲突是指：本来的LR(1)项集没有冲突，而合并具有相同核心的项集后有冲突
- 不可能引入归约-移入型冲突
 ➤ 假定合并后有移入-归约冲突，就是说：有项 $[A \rightarrow \alpha \cdot, a]$ 和项 $[B \rightarrow \beta \cdot a\gamma, b]$ 。显然，原来的项集中都有 $[B \rightarrow \beta \cdot a\gamma, c]$ 。而 $[A \rightarrow \alpha \cdot, a]$ 必然也在某个原来的项集中。这样，合并前的LR(1)项集已经存在移入-归约冲突
- 但是可能引起归约-归约冲突

自底向上的语法分析

- 练习题：给出如下文法的语法分析表，判断该文法是否SLR文法。

$$E \rightarrow E + E \mid E * E \mid (E) \mid id$$

自底向上的语法分析

- 练习题：给出如下文法的语法分析表，判断该文法是否SLR文法。
- 该文法不是 SLR(1) 文法

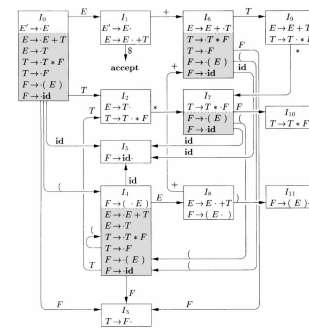
第二章 语法分析

冯洋

南京大学计算机学院
fengyang@nju.edu.cn

自底向上的语法分析

- 文法：(1) $E \rightarrow E + T$ (2) $E \rightarrow T$
 (3) $T \rightarrow T * F$ (4) $T \rightarrow F$
 (5) $F \rightarrow (E)$ (6) $F \rightarrow id$



[illegible]